CS 162 — Homework 2

Kaushik Vada

Problem 1

Answer: The number of threads that maximizes performance is $N = 6$.

Work. Let N be the number of threads. To make the math easy, normalize the 1-thread execution time to 100 time units.

- **SERIAL (S):** 20 units (does not change with N).
- **PARALLEL-COMPUTE (PC):** 75 units at $N = 1$ and scales ideally, so time is $\frac{75}{N}$.
- **PARALLEL-SYNC (PS):** the amount of PS work grows linearly with N, so

  $$\text{Work}_{PS} = 5N.$$

  PS is 60% parallelizable, so:

  - Serial portion: $0.4 \cdot 5N = 2N$.
  - Parallel portion: $0.6 \cdot 5N = 3N$. Spread across N threads, this takes $\frac{3N}{N} = 3$ time units.

  Therefore, $PS(N) = 2N + 3$.

Putting everything together, the total runtime is

$$T(N) = 20 + \frac{75}{N} + (2N + 3) = 23 + \frac{75}{N} + 2N.$$

To maximize performance we minimize $T(N)$. Differentiate with respect to N:

$$T'(N) = -\frac{75}{N^2} + 2 = 0 \Rightarrow N^2 = 37.5 \Rightarrow N \approx 6.12.$$

Since N must be an integer, check nearby values:

$$T(6) = 23 + \frac{75}{6} + 12 = 47.5, \qquad T(7) = 23 + \frac{75}{7} + 14 \approx 47.71.$$

So $N = 6$ gives the minimum time (and thus the best performance).

Problem 2

Answer:

- **Pipelined clock cycle time:** 800 ps.
- **Non-pipelined clock cycle time:** 1750 ps.
- **Speedup:** $\frac{1750}{800} = 2.1875$.

Work.

- **Pipelined clock:** the cycle time is limited by the slowest stage (here, MEM), so

$$T_{\text{pipe}} = \max(250, 350, 150, 800, 200) = 800 \text{ ps}.$$

- **Non-pipelined clock:** the cycle time is the sum of all stage delays:

$$T_{\text{non-pipe}} = 250 + 350 + 150 + 800 + 200 = 1750 \text{ ps}.$$

- **Speedup:**

$$\text{Speedup} = \frac{T_{\text{non-pipe}}}{T_{\text{pipe}}} = \frac{1750}{800} = 2.1875.$$

Problem 3

Branch Prediction Strategy

Answer: Strategy 3 (predict not taken).

Work. I computed the total CPI for each strategy.

Instruction mix: Arith (50%), L/S (20%), Branch (20%), Jump (10%).

Base CPI (ignoring branch penalties):

$$0.5(1) + 0.2(6) + 0.2(1) + 0.1(1) = 2.0.$$

Now add the branch penalty component (only applies to the 20% branch instructions):

- **Strategy 1 (always stall):** +1 cycle on every branch.

$$\Delta\text{CPI} = 0.2 \cdot 1 = 0.2 \Rightarrow \text{CPI} = 2.0 + 0.2 = \mathbf{2.2}.$$

- **Strategy 2 (predict taken):** +2 cycles if the branch is actually not taken (40%).

$$\Delta\text{CPI} = 0.2 \cdot (0.4 \cdot 2) = 0.16 \Rightarrow \text{CPI} = 2.0 + 0.16 = \mathbf{2.16}.$$

- **Strategy 3 (predict not taken):** +1 cycle if the branch is actually taken (60%).

$$\Delta\text{CPI} = 0.2 \cdot (0.6 \cdot 1) = 0.12 \Rightarrow \text{CPI} = 2.0 + 0.12 = \mathbf{2.12}.$$

Since Strategy 3 has the smallest CPI, it gives the fastest execution.

Problem 4

Branch Predictor Accuracy

1) **1-bit predictor (starts NT)**

Sequence: T, T, T, NT, NT.
Accuracy: 3 correct out of 5 branches $\Rightarrow$ 60%.

Branch	Prediction	Actual	Correct?
1	NT	T	No
2	T	T	Yes
3	T	T	Yes
4	T	NT	No
5	NT	NT	Yes

2) **2-bit predictor (start state choice)**

Pattern: T, T, NT, T, NT.
Worst start state: SNT (strongly not taken).

Reasoning (sketch): Starting in SNT means the predictor is biased toward NT early on, so it mispredicts repeatedly for this pattern and ends up with 0 correct predictions out of 5.

3) **2-bit predictor (infinite loop)**

Pattern: T, T, NT, NT, NT (repeated forever).
Start state: SNT.

Reasoning: If we track one iteration we get the hit/miss pattern (miss, miss, miss, hit, hit), which is 2 hits out of 5 branches. After the fifth branch the predictor returns to SNT, so this same pattern repeats every iteration. Therefore, the long-run accuracy is

$$\frac{2}{5} = 40\%.$$

Problem 5

Hazards & Stalls

Code sequence:

```
add $3, $1, $2
lw $1, 0($4)
and $5, $3, $4
and $6, $1, $2
or $1, $3, $6
sw $1, 4($4)
lw $2, 4($4)
```

1) Data hazards (RAW)

Producer	Consumer	Reg.	Type
1 (add)	3 (and)	$3	RAW
1 (add)	5 (or)	$3	RAW
2 (lw)	4 (and)	$1	RAW
4 (and)	5 (or)	$6	RAW
5 (or)	6 (sw)	$1	RAW

2) Stalls without forwarding

Assuming the standard MIPS pipeline timing (registers read in the second half of ID and written in the first half of WB), you generally need two independent instructions between a producer and consumer to avoid stalling.

- Between 1–2: 0 stalls.
- Between 2–3: 1 stall (`add` produces $3, and instruction 3 uses $3 with only one instruction in between).
- Between 3–4: 0 stalls.
- Between 4–5: 2 stalls (instruction 4 produces $6 and instruction 5 immediately uses it).
- Between 5–6: 2 stalls (instruction 5 produces $1 and instruction 6 immediately uses it).
- Between 6–7: 0 stalls.

Total stalls:

$$0 + 1 + 0 + 2 + 2 + 0 = \mathbf{5}.$$

3) Stalls with forwarding

With full forwarding, most ALU-to-ALU and ALU-to-store hazards are handled by bypass paths. The main case that still stalls is a *load-use* hazard where an instruction uses a loaded value in the very next cycle.

Here, instruction 2 loads $1 and instruction 4 uses $1, but there is one instruction between them, so forwarding is sufficient and no stall is needed.

Total stalls: **0**.